

Sprawozdanie z projektu: Automatyczne generowanie bloga przy użyciu Gemini API i bazy danych Neon.tech

Yurii Vorotnyy i Łukasz Lipski

14 kwietnia 2025

1 Wprowadzenie

Projekt realizuje system automatycznego tworzenia treści blogowych. Codziennie o godzinie 12:00 uruchamiane jest zadanie CRON, które:

1. Wybiera losowo temat z listy wcześniej przygotowanych tematów.
2. Oznacza go jako wykorzystany.
3. Wysyła temat do API Gemini z odpowiednim promptem.
4. Odbiera wygenerowaną treść bloga w formacie Markdown.
5. Zapisuje ją do bazy danych Neon.tech.

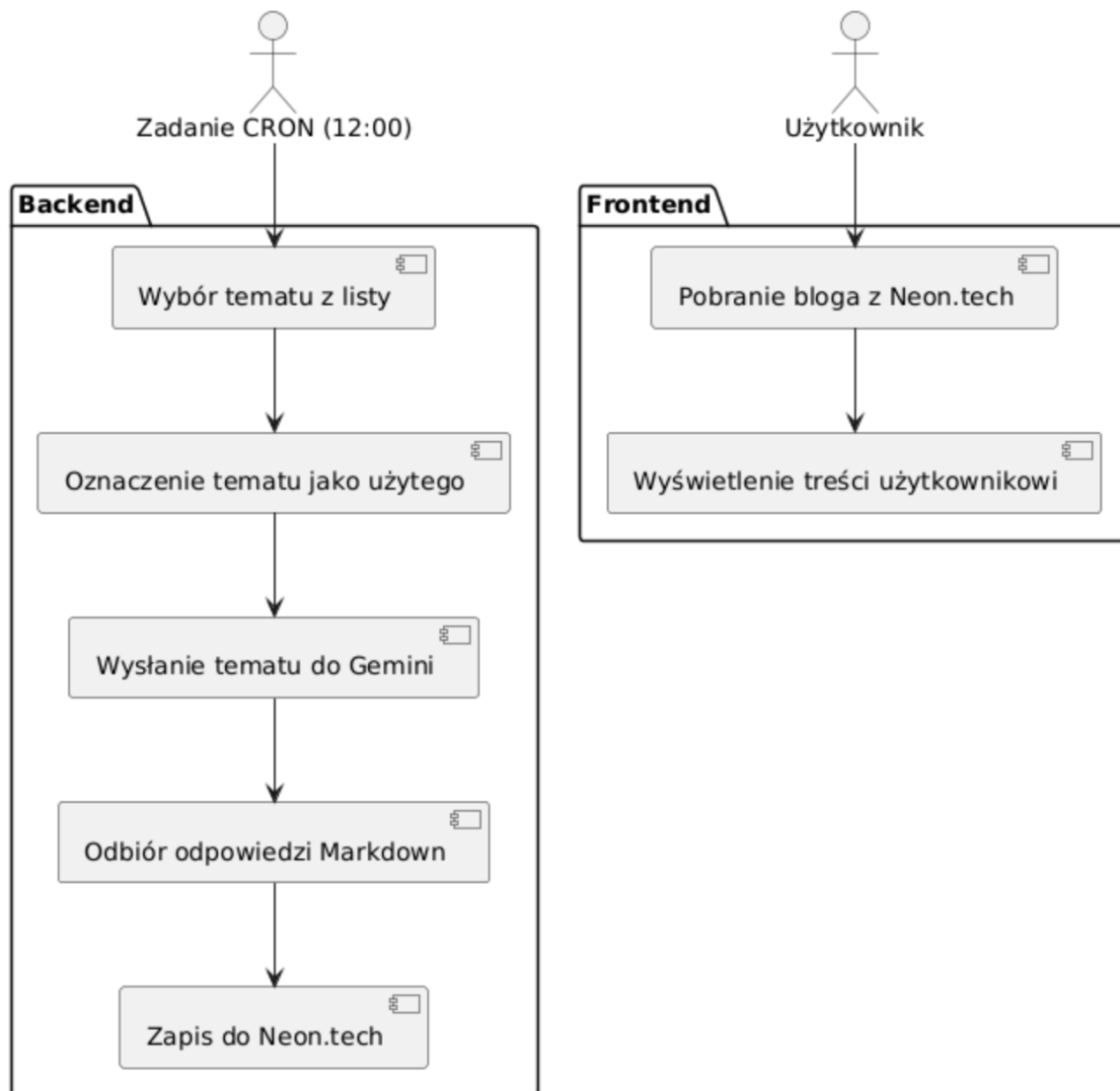
Użytkownik odwiedzający frontend otrzymuje zawsze najnowszą treść pobraną z bazy danych. Rozwiązanie to automatyzuje cały proces tworzenia i publikowania treści blogowych, eliminując potrzebę ręcznej ingerencji.

2 Architektura

2.1 Opis przepływu danych

- Codziennie o 12:00 zadanie CRON uruchamia backendowy proces publikacji.
- Backend wybiera temat, oznacza go jako użyty, wysyła zapytanie do OpenAI i zapisuje wynik w Markdownie do Neon.tech.
- Frontend użytkownika pobiera dane z bazy i wyświetla je.

2.2 Diagram komponentów (UML)



Rysunek 1: Diagram komponentów systemu

3 Prowadzenie projektu

3.1 Work Breakdown Structure (WBS)

- Zaprojektowanie architektury systemu (0.5 dnia)
- Przygotowanie listy tematów blogowych (1 godzina)
- Implementacja harmonogramu CRON (1 godzina)

- Wybór tematu i oznaczanie go jako wykorzystany (15 min)
- Integracja z OpenAI API (1 godzina)
- Integracja z bazą Neon.tech (1 dzień)
- Budowa frontendowego odczytu i wyświetlania (1 dzień)
- Testowanie systemu i korekty błędów (1 dzień)

Łączny szacowany czas realizacji: 4 dni robocze.

4 Prezentacja działającego programu

```
def get_unused_topic():
    """
    Reads the topics.txt file and finds an unused topic.
    """
    # Read all topics
    with open(TOPICS_FILE, "r", encoding="utf-8") as f:
        all_topics = [line.strip() for line in f.readlines() if line.strip()]

    # Read used topics
    if os.path.exists(USED_TOPICS_FILE):
        with open(USED_TOPICS_FILE, "r", encoding="utf-8") as f:
            used_topics = set(line.strip() for line in f.readlines())
    else:
        used_topics = set()

    # Find the first unused topic
    for topic in all_topics:
        if topic not in used_topics:
            return topic

    return None # No unused topics left


def generate_blog_post(topic):
    """
    Generates a blog post using Google Gemini AI.
    Returns a tuple containing (slug, date, title, content).
    """
    client = genai.Client(api_key=GEMINI_TOKEN)
    response = client.models.generate_content(
        model="gemini-2.0-pro-exp-02-05",
        contents=f"""
        Stwórz zoptymalizowany pod kątem SEO artykuł blogowy w formacie Markdown.
        Zwróć wynik w formacie:

        slug-url(bez słowa markdown, z minusami zamiast spacji, zgodny z składnią path URL) | {datetime.now().date()} | Tytuł
        artykułu | Treść artykułu (bez nagłówka tytułowego)

        Temat artykułu: {topic}

        Wytoczne SEO:
        - Długość: 1500-2500 słów
        - Hierarchia nagłówków markdown
        - Słowa kluczowe wplecione naturalnie
        - Struktura dla Featured Snippets
        - Czytelność UX: krótkie akapity, listy punktowane, wyróżnienia
        - Styl: Profesjonalny, ale angażujący
        """
    )
```

Rysunek 2: Wybór tematu i generacja treści

```

def insert_blog_post(table_name, columns, slug, title, content):
    """
    Inserts the generated blog post into the specified table.
    """
    conn = connection_pool.getconn()
    cur = conn.cursor()

    # Quote the table name to handle case-sensitivity
    table_name = f'"{table_name}"'

    required_fields = ["id", "slug", "title", "date", "content", "createdAt", "updatedAt"]
    available_fields = [col for col in required_fields if col in columns]

    quoted_columns = [f'"{col}"' for col in available_fields]

    insert_query = f"""
    INSERT INTO {table_name} ({', '.join(quoted_columns)})
    VALUES ({', '.join(['%s'] * len(available_fields))})
    RETURNING id;
    """

    generated_id = cuid.cuid()

    values = {
        "id": generated_id,
        "slug": slug,
        "title": title,
        "date": datetime.now().date(),
        "content": content,
        "createdAt": datetime.now(),
        "updatedAt": datetime.now()
    }

    insert_values = [values[col] for col in available_fields]

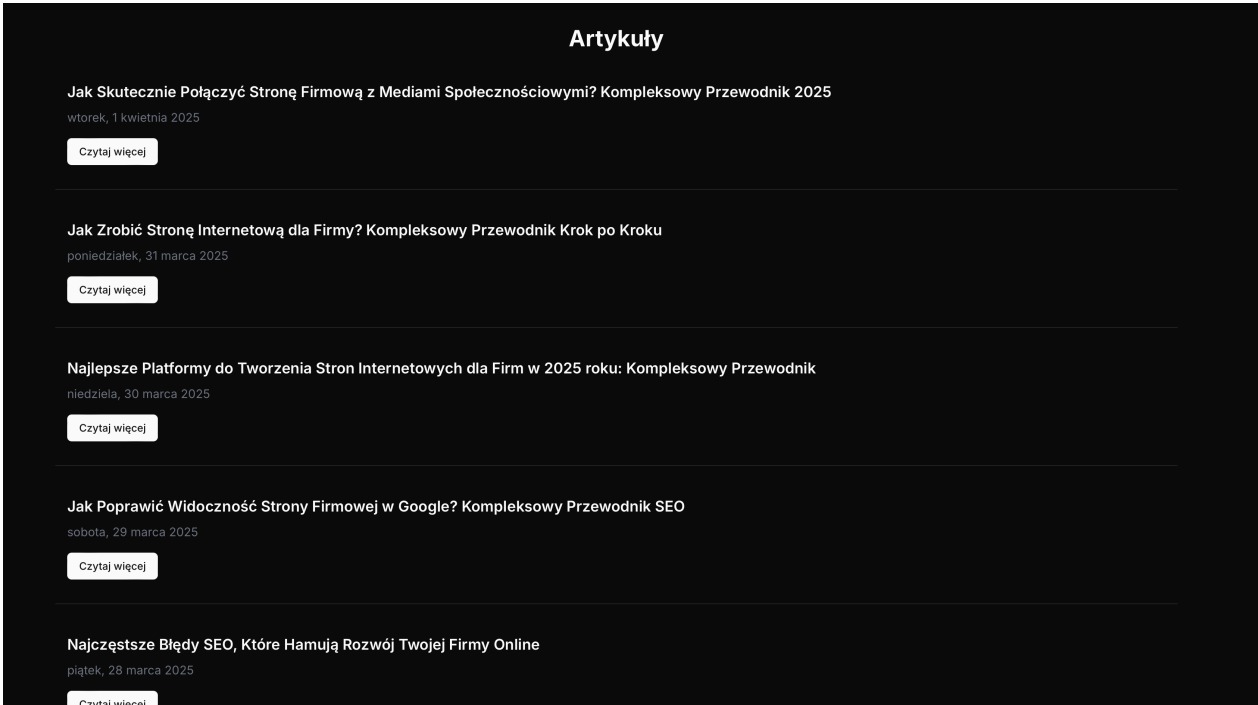
    cur.execute(insert_query, insert_values)
    post_id = cur.fetchone()[0]

    conn.commit()
    cur.close()
    connection_pool.putconn(conn)

    print(f"Blog post successfully inserted into '{table_name}' with ID: {post_id}")
    return post_id

```

Rysunek 3: Zapis wygenerowanej treści w Markdownie do Neon.tech



Rysunek 4: Wyświetlenie blogów na stronie

Jak Skutecznie Połączyć Stronę Firmową z Mediami Społecznościowymi? Kompleksowy Przewodnik 2025

wtorek, 1 kwietnia 2025

W dzisiejszym cyfrowym świecie **obecność online** to absolutna podstawa sukcesu każdej firmy. Posiadanie profesjonalnej strony internetowej to dopiero początek. Równie kluczowe jest **zintegrowanie jej z mediami społecznościowymi**, które stały się nieodłącznym elementem życia milionów potencjalnych klientów. Ale jak to zrobić skutecznie? Jak sprawić, by strona i profile social media działały synergicznie, wzmacniając wzajemnie swój zasięg i efektywność?

Ten kompleksowy przewodnik pokaże Ci krok po kroku, **jak połączyć stronę firmową z mediami społecznościowymi**, aby zmaksymalizować widoczność Twojej marki, zwiększyć zaangażowanie użytkowników i napędzić ruch na stronie. Omówimy korzyści, konkretne metody integracji, najlepsze praktyki i narzędzia, które Ci w tym pomogą.

Dlaczego Integracja Strony Firmowej z Social Media Jest Kluczowa?

Zanim przejdziemy do technicznych aspektów, zastanówmy się, dlaczego **połączenie strony internetowej z profilami społecznościowymi** jest tak istotne dla Twojego biznesu. Korzyści jest wiele i obejmują one kluczowe obszary marketingu online:

- **Zwiększona Widoczność i Zasięg:** Media społecznościowe to potężne platformy do docierania do szerokiego grona odbiorców. Łącząc je ze stroną, tworzysz więcej punktów styku z potencjalnymi klientami.
- **Budowanie Spójnego Wizerunku Marki:** Integracja pozwala na utrzymanie jednolitej komunikacji i identyfikacji wizualnej we wszystkich kanałach online, co buduje zaufanie i rozpoznawalność.
- **Napędzanie Ruchu na Stronie WWW:** Aktywne profile społecznościowe, które regularnie linkują do wartościowych treści na Twojej

Rysunek 5: Wyświetlenie treści bloga na stronie

5 Wdrażanie aplikacji

Aplikacja wdrażana jest z wykorzystaniem GitHub Actions, które automatyzują budowanie, testowanie i publikację. Proces wygląda następująco:

1. Kod źródłowy znajduje się w repozytorium GitHub.
2. Zmiany wprowadzane w repozytorium uruchamiają GitHub Actions.
3. Automatyczne testy weryfikują poprawność działania.
4. Po akceptacji, backend automatycznie publikuje treść do bazy Neon.tech.
5. Frontend odświeża dane bezpośrednio z bazy, dzięki czemu użytkownicy od razu widzą nowe wpisy.

W przypadku wystąpienia jakichkolwiek problemów (np. brak dostępnych tematów lub błędy w generacji treści), system automatycznie wysyła wiadomość e-mail do administratora, aby natychmiastowo podjął działania naprawcze.

Blog Publish Error

Date: 2025-03-12 01:50:43

Error Type: ValueError

Message: This is a test error! Something went wrong in the blog generation process.

Traceback:

```
Traceback (most recent call last):  
  File "/Users/hypocristis/Documents/Projekty/Blog Generation/Blog_Generation/main.py", line 233, in  
    raise ValueError("This is a test error! Something went wrong in the blog generation process.")  
ValueError: This is a test error! Something went wrong in the blog generation process.
```

Rysunek 6: Wiadomość do administratora